

Report: Sentence Segmentation in Urdu and Sentence-Piece Tokenizer

Adnan Yousaf

19i-1742

Introduction

Urdu, being a complex language, with a rich morphology and requires precise treatment of sentence boundaries and punctuation, text processing with it presents special obstacles. In this project, an encoder-decoder system and a basic custom tokenizer were constructed especially for Urdu text. This study aims to provide an explanation of the code's architecture and operation, encompassing the many techniques for text segmentation, tokenization, and encoding.

1. Reading and Normalizing the Text

The text corpus is read from a file (urdu-corpus.txt) in the first section of the code. Next, excess spaces are eliminated from the text by normalizing it. This is an important preprocessing step since irregular spacing frequently causes sentence boundaries to be misinterpreted when working with huge datasets. Regular expressions are used by the `normalize_spaces` function to replace numerous spaces with a single space.

```
def normalize_spaces(text):  
    text = re.sub(r'\s+', ' ', text)  
    text = text.strip()  
    return text
```

This simple function ensures that the text is cleaner and more consistent, which is crucial for further processing.

2. Urdu Sentence Segmentation

Sentence segmentation is handled by the code by recognizing common punctuation and Urdu sentence-ending terms, like "تھی", "ہے", and the Urdu full stop ".". Based on these signals, the segmentation procedure makes sure that the sentences are separated appropriately.

```
def urdu_sentence_segmentation(text):  
    end_words = ['ہے', 'ہیں', 'تھا', 'تھی', 'گی', 'گئے', 'دی', 'کیا', 'رہا']  
    # Adds space after punctuation if missing  
    text = re.sub(r'([!?.])([^\s])', r'\1 \2', text)  
    # Sentence splitting logic  
    sentences = []  
    ...  
    return sentences
```

The process of segmentation involves going over the text repeatedly and dividing it into sentences using either sentence-ending or punctuation terms. Accurately dividing text into understandable units depends on this.

3. Tokenization

The tokenization step is responsible for breaking down each sentence into smaller units called tokens. The code defines a custom tokenizer (custom_tokenize) that handles Urdu punctuation marks and spaces.

```
def custom_tokenize(sentence):
    tokens = []
    current_token = ''
    for char in sentence:
        if char in urdu_punctuation or char.isspace():
            if current_token != '':
                tokens.append(current_token)
                current_token = ''
            if not char.isspace():
                tokens.append(char)
        else:
            current_token += char
    if current_token != '':
        tokens.append(current_token)
    return tokens
```

This tokenizer effectively separates words from punctuation and creates a list of tokens that can be used for further encoding or analysis.

4. Encoding and Decoding

The code not only tokenizes but also encodes and decodes sentences. This is helpful for formatting text so that it may be used with machine learning models or for additional processing.

- **Encoder:** Converts a list of sentences into tokens using the tokenizer.

```
def custom_encode(sentences):
    encoded_sentences = []
    for sentence in sentences:
        tokens = custom_tokenize(sentence)
        encoded_sentences.append(tokens)
    return encoded_sentences
```

- **Decoder:** Converts the tokens back into sentences, reconstructing the original text.

```
def custom_decode(encoded_sentences):
    decoded_sentences = []
    for tokens in encoded_sentences:
        ...
    return decoded_sentences
```

While the decoder piecemeals the sentence back together from the tokenized version, the encoder splits the text into a token sequence. This ensures that no information is lost during the encoding process and enables verification of whether the encoded and decoded texts are the same.

5. BPE Tokenizer Implementation

Byte Pair Encoding (BPE) algorithm is frequently used in natural language processing to build a vocabulary from a corpus of texts. This code trains a basic BPE tokenizer with a set vocabulary size on the corpus.

```
class BPETokenizer:
    def train(self, corpus):
        tokens = [' '.join(word) + ' </w>' for word in corpus.split(' ')]
        vocab = Counter(corpus)
        while len(self.vocab) < self.vocab_size:
            pairs = self.get_stats(vocab)
            ...
        self.tokens = list(self.vocab.keys())
```

The BPETokenizer joins frequently occurring character pairings to produce a set of tokens. Until the required vocabulary size is obtained, this process is repeated. By employing subword units rather of entire words, this technique enables more effective text encoding, which is particularly helpful for languages like Urdu, which have a large number of morphological changes.

6. Evaluation of Segmentation

The evaluation function used in this code basically compares the predicted sentence boundaries with a manually created reference. This helps in measuring the precision, recall,

and F1 score of the segmentation process.

Conclusion

These algorithms handle tokenization, encoding, decoding, and sentence segmentation, exhibiting a comprehensive approach to Urdu text processing. In particular, the BPE tokenizer is helpful when working with big corpora that have a small vocabulary. The code is straightforward, but it may be extended to handle more difficult jobs like text creation models or machine translation.

More advanced sentence-ending recognition, the use of machine learning models for tokenization, or an extension of the BPE tokenizer to handle bigger datasets and more in-depth examination of token frequencies in natural language are possible future enhancements.